Lesson 12

Normalization and Initialization

정규화와 초기화: 깊은 신경망이 안정적으로 배우는 비결

활성값 · ReLU · 가중치 초기화(Xavier, He) · Batch Norm · Layer Norm

평균 0 **분산 1** **안정!**

튜링랩 프로그래밍학원
TuringLab. Programming Academy

TuringLab.

OVERVIEW

오늘의 로드맵: 3교시 (50분 수업 + 10분 휴식) × 3

교시	주제	직접 해보는 것
1교시	활성값 · 비선형성 · XOR · ReLU와 그 친구들	Step 1 선형 붕괴 확인, XOR 풀기
2교시	가중치 초기화: 분산, Xavier, He, 신호 폭발·소실	Step 2 10층 신호 크기 실험
3교시	미니배치 · Batch Norm · Layer Norm · 종합 실험	Step 3~4 정규화 직접 구현, 학습 비교
마무리	실습 문제 5개 + 솔루션, 원본 자료 검토 결과	손계산 · 구현 · 예측 후 실험

준비물: Python 3.9+, NumPy, (Step 3·4) PyTorch, matplotlib | 코드는 lesson12/code 폴더

GOAL

이 강의가 끝나면 할 수 있는 일

1. **설명하기**
   활성값, 비선형성, 내부 공변량 변화, 정규화가 왜 필요한지 내 말로 설명한다

2. **계산하기**
   Xavier/He 초기화의 분산을 계산하고, 배치 정규화를 손으로 계산한다

3. **구현하기**
   BatchNorm / LayerNorm을 NumPy로 직접 만들고 PyTorch와 결과를 비교한다

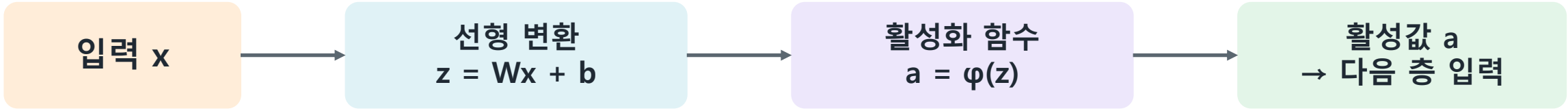
4. **고르기**
   상황(활성화 함수, 배치 크기, 모델 종류)에 맞는 초기화·정규화를 선택한다

1교시

활성값과 비선형성

신경망은 왜 '구부러지는' 함수가 필요한가 · ReLU 다시 보기

신경망 한 층이 하는 일



- W, b : 가중치와 편향 (학습으로 바뀌는 숫자들)
- φ : ReLU, Sigmoid, Tanh 같은 활성화 함수
- 활성값(activation) a : 뉴런이 '활성화 함수를 통과한 뒤' 내놓는 값
- 활성값은 단순한 숫자가 아니라 다음 층으로 전달되는 신호(signal)

이번 강의의 핵심 질문: 층이 깊어질수록 이 활성값의 크기와 분포가 흐트러지는데, 어떻게 안정적으로 유지할까?

선형(linear)과 비선형(non-linear)

선형 함수의 조건

f(ax + by) = a·f(x) + b·f(y)

'더하기와 곱하기'만으로 이루어진 함수.
공간을 회전·이동·늘리기만 한다.

비선형 함수 = 위 성질을 만족하지 않는 함수

ReLU: f(x) = max(0, x)
Sigmoid, Tanh

공간을 접고, 자르고, 구부린다.

선형 층만 쌓으면 어떻게 될까?

$$y = W_3(W_2(W_1x)) = (W_3W_2W_1)x = Wx$$

→ 아무리 깊어도 결국 하나의 선형 변환 (얇은 층 1개와 같다)

활성화 함수가 없다면: 깊은 신경망 = 얕은 선형 모델

직접 확인: 선형 2층 = 선형 1층

step1_nonlinear.py

```
W1 = rng.normal(size=(3, 4))    # 1층 가중치 (3 -> 4)
W2 = rng.normal(size=(4, 2))    # 2층 가중치 (4 -> 2)
x  = rng.normal(size=(5, 3))    # 샘플 5개

deep = (x @ W1) @ W2            # 두 층을 차례로 통과
one  = x @ (W1 @ W2)            # 미리 곱해 둔 1층
np.allclose(deep, one)          # True !

relu = lambda z: np.maximum(0, z)
deep_relu = relu(x @ W1) @ W2   # 중간에 ReLU를 끼움
np.allclose(deep_relu, one)     # False
```

코드 읽는 법

① x @ W : 행렬곱. 한 층의 선형 변환

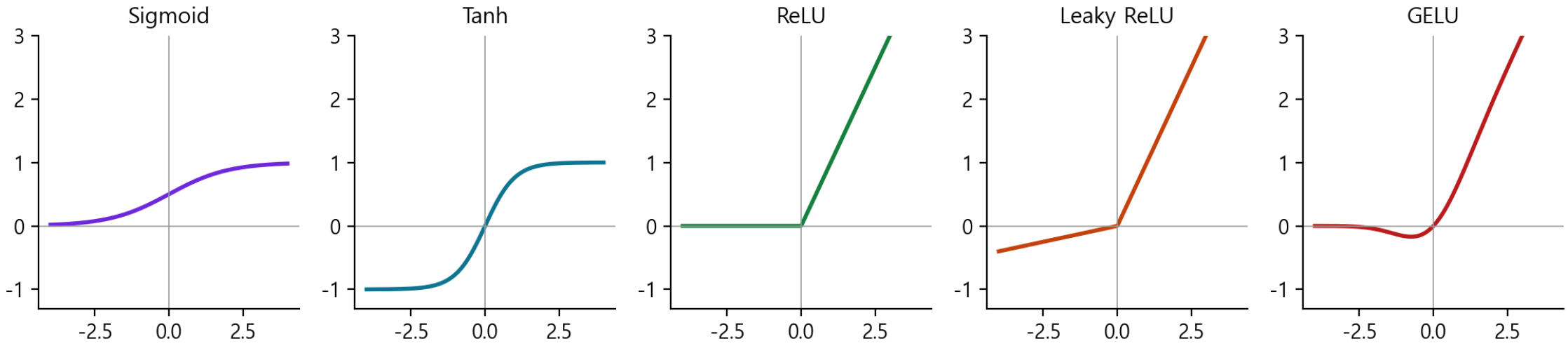
② (x@W1)@W2 와 x@(W1@W2)는 같다 (결합법칙)

③ ReLU를 끼우면 하나로 합칠 수 없다 → 표현력이 늘어난다

④ 결과: True / False 로 직접 확인

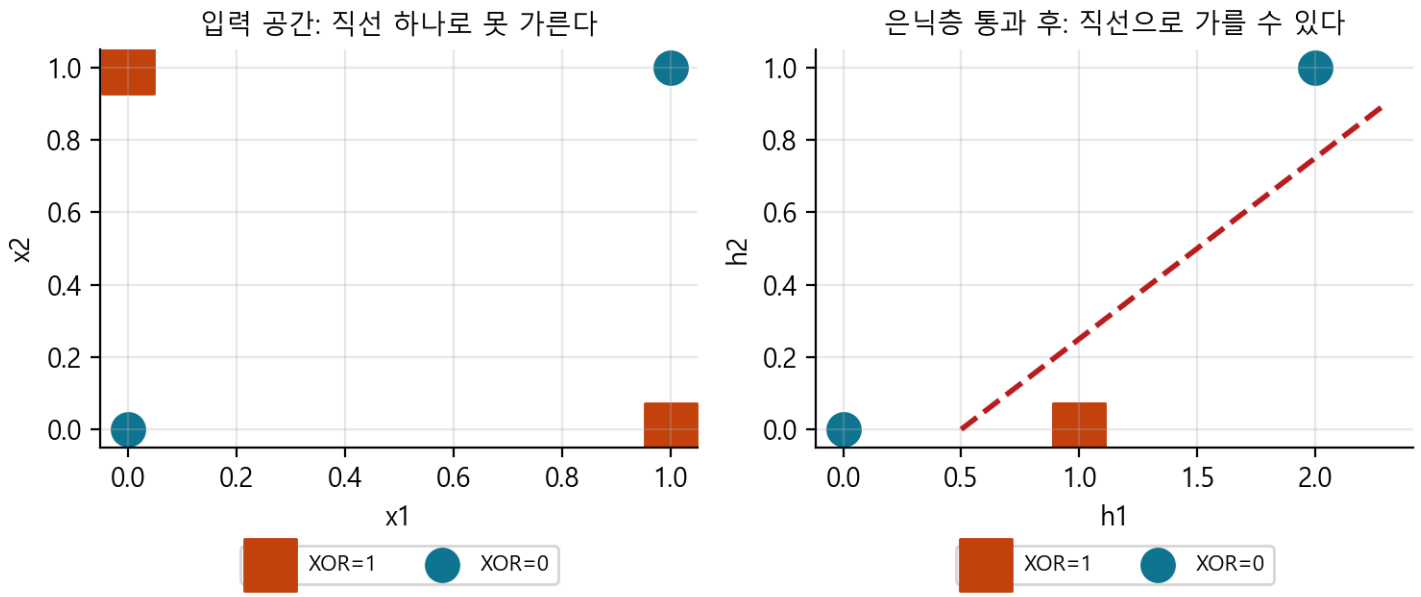
실행: python step1_nonlinear.py

대표적인 활성화 함수들



- Sigmoid(0~1), Tanh(-1~1): 부드러운 S자. 양 끝에서 평평해져 기울기가 0에 가까움
- ReLU: 0 이하는 0, 양수는 그대로. 가장 널리 쓰임
- Leaky ReLU, GELU: ReLU의 약점을 보완한 후손들 (뒤에서 다룸)

왜 비선형성이 꼭 필요한가? XOR 문제



XOR (배타적 논리합)

서로 다르면 1, 같으면 0

(0,0)→0 (0,1)→1

(1,0)→1 (1,1)→0

직선 하나(선형 모델)로는 절대 두 그룹을 가를 수 없다.

비선형 활성화가 들어가면

중간 표현 공간을 비틀고 → 입력을 다른 공간으로 옮겨서 → 선형으로 가를 수 있게 된다.
현실의 문제(이미지·음성·언어)는 대부분 선형으로 표현할 수 없다.

비선형성 = 공간을 '구부리는' 것

선형 변환

공간을 회전, 이동, 스케일링만 한다.

종이를 돌리고 옮기고 늘리는 것과 같다.
구겨지지 않는다.

비선형 변환

공간을 접고, 자르고, 구부린다.

종이를 접어서 서로 떨어진 점들을 한곳에 포개거나
,
겹쳐 있던 점들을 갈라놓을 수 있다.

- ReLU는 '음수 부분을 0으로 접는' 가장 단순한 비선형 변환
- 접힌 공간을 여러 번 겹치면 아주 복잡한 모양도 표현 가능
- 활성화 함수 = 비선형성을 '의도적으로 주입'하는 장치

직접 확인: 은닉층 + ReLU로 XOR 풀기

step1_nonlinear.py

```
X = [[0,0],[0,1],[1,0],[1,1]]    # XOR 입력 4가지
Wh = [[1,1],[1,1]]; bh = [0,-1] # 은닉 뉴런 2개
Wo = [1, -2]                     # 출력층

h   = relu(X @ Wh + bh)  # 은닉층: (x1+x2), (x1+x2-1)
out = h @ Wo             # 출력: h1 - 2*h2
# out = [0, 1, 1, 0]  <- 정답과 일치!

# 선형 모델(최소제곱 직선)의 최선
# -> [0.5, 0.5, 0.5, 0.5]  XOR 실패
```

코드 읽는 법

① 은닉층: 두 입력을 합친 값 x1+x2 와, 거기서 1을 뺀 값을 ReLU에 통과

② (0,1)과 (1,0)이 같은 점 (1,0)으로 포개진다 ← 공간이 접혔다!

③ 출력층의 직선 하나로 이제 분리 가능

④ 선형 모델은 모든 입력에 0.5 → 전혀 구분 못 함

가중치는 학습이 아니라 손으로 구함: '구조'만으로 풀 수 있음을 보여 주는 예

ReLU 다시 보기

$$\mathrm{ReLU}(x) = \max(0, x)$$

- 입력이 양수면 그대로 출력
- 입력이 0 이하면 0 출력
- 아주 단순하지만, 이 단순함이 학습을 크게 안정시킨다

입력 x	ReLU(x)	기울기
-3	0	0
-0.5	0	0
0.5	0.5	1
3	3	1

입력 → 출력 예시

x = [-2, -1, 0, 1, 2] → ReLU(x) = [0, 0, 0, 1, 2]
음수는 모두 0으로 '꺼지고', 양수만 살아남는다.

왜 ReLU가 중요한가? (장점 3가지)

① 기울기 소실 완화

sigmoid/tanh는 입력이 크면 기울기가 거의 0이 되어 깊은 층에서 학습이 안 된다.

ReLU는 양수 영역에서 기울기가 1이라 깊은 층까지 기울기가 잘 전달된다.

② 계산이 매우 빠름

max 연산 하나면 끝.

미분도 간단(0 또는 1)해서 GPU에서 특히 효율적이다.

③ 희소성(Sparsity) 유도

입력의 절반 정도가 0이 되어 뉴런이 선택적으로 활성화된다.

일종의 정규화 효과를 주어 일반화에 도움이 된다.

※ '희소성(sparsity)': 대부분의 값이 0인 성질 (원본의 sparity는 오타)

ReLU의 단점과 그 후손들

① Dying ReLU (죽은 ReLU)

입력이 계속 음수이면 출력이 영원히 0이 되고 기울기도 0이 되어, 그 뉴런은 더 이상 학습되지 않는다.

② 출력 분포가 한쪽으로 치우침

음수를 모두 0으로 만들어 출력 평균이 양수 쪽으로 쏠릴 수 있다.

BatchNorm 같은 정규화와 함께 쓰면 훨씬 안정적이다.

해결책(변형)	아이디어
Leaky ReLU	음수에서도 작은 기울기 허용: f(x) = 0.01x
Parametric ReLU	그 음수 기울기를 학습으로 정함
ELU	음수 영역을 부드러운 곡선으로
GELU	부드러운 ReLU. Transformer(BERT, GPT)에서 표준

1교시 정리 + 생각해 보기

- **활성값 = 활성화 함수를 통과한 뒤 다음 층으로 전달되는 신호**
- 활성화 함수가 없으면 아무리 깊어도 선형 모델 1개와 같다 (Step 1 확인)
- 비선형성은 공간을 접고 구부려 XOR 같은 문제를 가능하게 한다
- ReLU: 빠르고 기울기가 잘 흐르지만 Dying ReLU와 치우친 출력이 단점

생각해 보기 (10분 휴식 전)

Q. 층을 20개 쌓았더니, 앞쪽 층의 가중치가 조금만 바뀌어도 뒤쪽 층이 받는 입력의 '크기와 분포'가 확 달라진다면 어떤 일이 벌어질까?

→ 다음 시간(2~3교시)의 주제로 이어집니다.

2교시

가중치 초기화 (Weight Initialization)

Xavier · He · 신호가 폭발하거나 사라지지 않게

왜 초기화가 중요한가?

학습이 시작되기 전, 가중치는 보통 무작위 값으로 정한다. 그런데 '아무 무작위'면 안 된다.

너무 크게 → 폭발

층을 지날수록 값이 기하급수로 커진다.

활성값·기울기 폭발

너무 작게 → 소실

층을 지날수록 값이 0에 가까워진다.

활성값·기울기 소실 → 학습이 거의 안 됨

적절하게 → 안정

층마다 값의 크기(분산)가 비슷하게 유지되어 학습이 잘 된다.

초기화의 목표: 모든 층에서 활성값과 기울기의 분산이 적절하게 통제되도록 하는 것.

준비 ①: 입력 유닛 n_in, 출력 유닛 n_out

입력 유닛 (Input Units, n_in)

한 층이 '받아들이는' 값의 개수
= 해당 층의 입력 벡터 차원 수

예) 입력 벡터가 $[x_1, x_2, x_3]$ → n_in = 3
예) 이전 층의 뉴런이 128개 → 다음 층의 n_in = 128

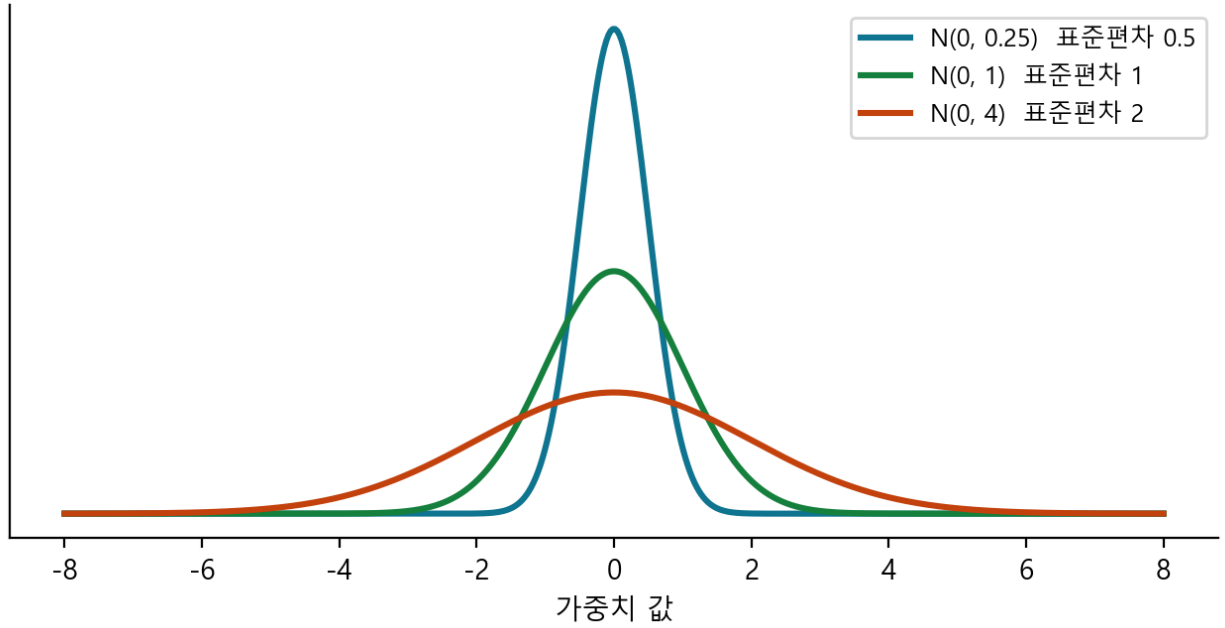
출력 유닛 (Output Units, n_out)

한 층이 '내보내는' 값의 개수
= 해당 층의 출력 벡터 차원 수

예) Dense layer(완전연결층)가 64개 뉴런 → n_out = 64
예) 10개 클래스 분류의 마지막 층 → n_out = 10



준비 ②: 정규분포 N(0, σ²)와 분산



$W_{ij} \sim N(0, \sigma^2)$: 'W_{ij}는 이 분포에서 뽑는다'는 뜻 (~ 는 같다(=)가 아니라 '확률적으로 뽑힌다')

N(0, σ²) 읽는 법

평균이 0이고 분산이 σ²인 정규분포.
분산(σ²): 값이 평균에서 얼마나 퍼져 있나
표준편차(σ) = √분산

예: N(0, 4)

평균 = 0, 분산 = 4
표준편차 = √4 = 2
→ 값 대부분이 0을 중심으로 ±2 범위에 분포

왜 분산이 n_in에 맞춰져야 할까? (직관 유도)

출력 뉴런 하나는 n_in개의 입력을 곱해서 더한다:

$$y = w_1x_1 + w_2x_2 + \ldots + w_n\, x_n \qquad (n = n_in)$$

서로 독립이고 평균이 0이면, 분산은 항목별로 더해진다:

$$\mathrm{Var}(y) = n_in \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(x)$$

- 층을 지나도 분산을 유지하려면 Var(y) = Var(x)
- → n_in · Var(w) = 1 → Var(w) = 1 / n_in
- 입력이 많을수록(n_in ↑) 각 가중치는 더 작아야 합이 폭주하지 않는다

Xavier 초기화 (자비어 / Glorot 초기화)

$$W_{ij} \sim N(0, 2 / (n_{in} + n_{out}))$$ (12.9)

- tanh처럼 0을 중심으로 대칭인 활성화 함수를 위해 설계
- 순전파(앞으로 가는 신호)와 역전파(뒤로 가는 기울기)가 모두 증폭·약화되지 않도록 균형을 맞춘다
- 순전파 조건은 1/n_in, 역전파 조건은 1/n_out → 둘을 절충(조화 평균)한 것이 2/(n_in + n_out)
- n_in = n_out 이면 정확히 1/n_in 과 같다

자주 보이는 변형

정규분포형: Var(W) = 2/(n_in+n_out) | 균등분포형: W ~ U(−a, a), a = √(6/(n_in+n_out)) | Var(W) = 1/n_in 은 LeCun 초기화

He 초기화 (카이밍 히 / Kaiming 초기화)

$$W_{ij} \sim N(0, 2 / n_{in}) \quad (12.10)$$

- ReLU처럼 음수를 0으로 만드는 비대칭 활성화 함수를 위해 설계
- ReLU는 출력의 절반을 0으로 만든다 → 신호의 분산이 절반으로 줄어든다
- 그래서 Xavier(1/n_in)보다 분산을 2배(2/n_in)로 키워 보상한다
- ReLU가 희소성을 만들어도 순전파·역전파의 분산이 유지된다

Xavier : tanh 계열

Var(W) = 2/(n_in + n_out)

He : ReLU 계열

Var(W) = 2/n_in (Xavier보다 큼)

실험 준비: 10층 신경망에 신호를 흘려 보내자

step2_init.py

```
def forward_stds(act, std_fn, layers=10, width=512, n=1000):
    a = rng.normal(size=(n, width))      # 입력: 평균 0, 표준편차 1
    stds = [a.std()]
    for _ in range(layers):
        std = std_fn(width, width)        # 초기화 규칙으로 표준편차 결정
        W = rng.normal(0, std, size=(width, width))    # W ~ N(0, std²)
        a = act(a @ W)                    # 선형 변환 + 활성화 함수
        stds.append(a.std())              # 이 층 활성값의 표준편차 기록
    return stds

xavier = lambda n_in, n_out: np.sqrt(2 / (n_in + n_out))
he     = lambda n_in, n_out: np.sqrt(2 / n_in)
```

코드 읽는 법

① 폭 512인 층 10개. 입력은 표준정규분포

② 가중치는 N(0, std²)에서 뽑는다 (std = 초기화 규칙)

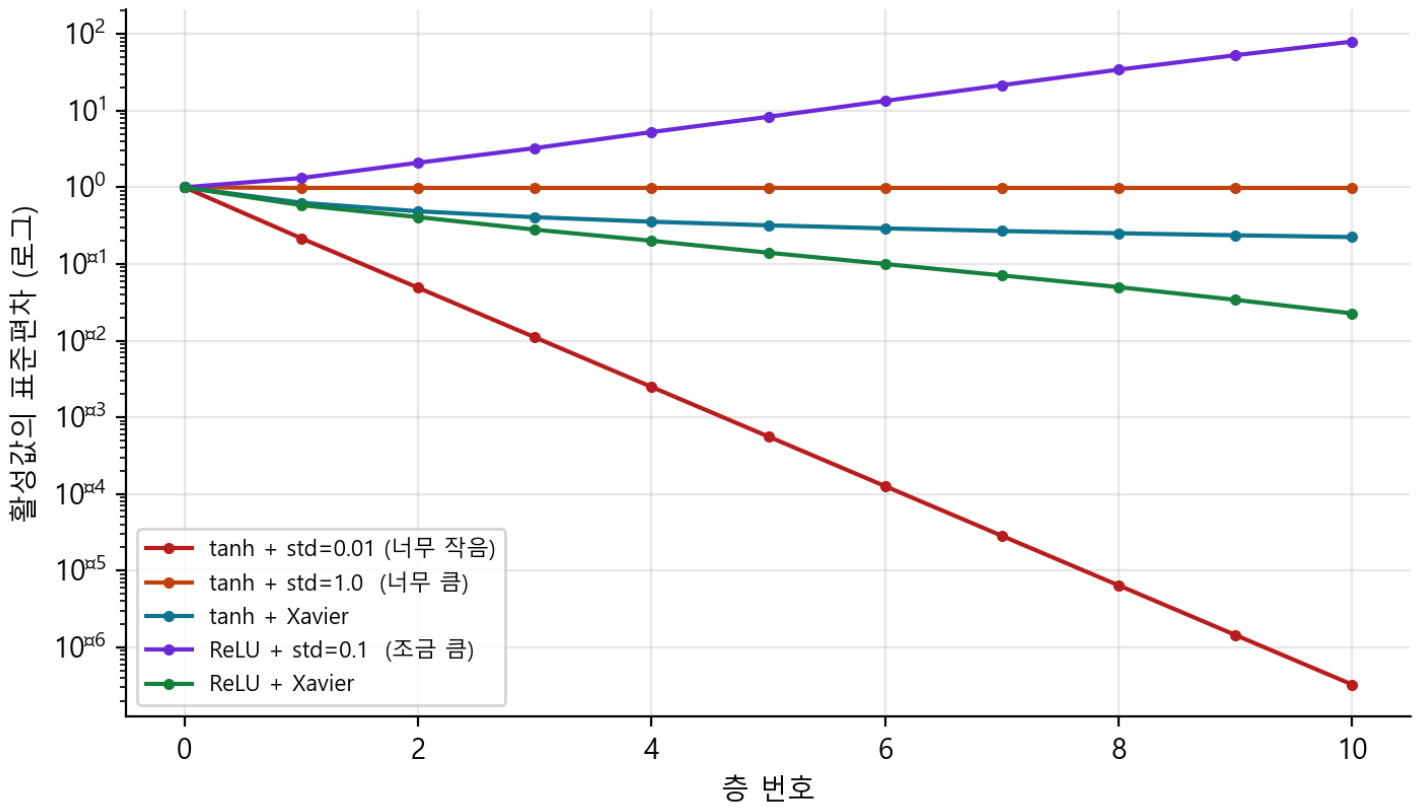
③ 층을 지날 때마다 활성값의 표준편차를 기록

④ std_fn만 바꿔서 '초기화 방식'을 비교한다

⑤ 표준편차가 1 근처 유지 = 안정, 0이나 큰 값으로 가면 소실/폭발

실행: python step2_init.py

실험 결과 ①: 층마다 활성값의 크기 (로그 눈금)



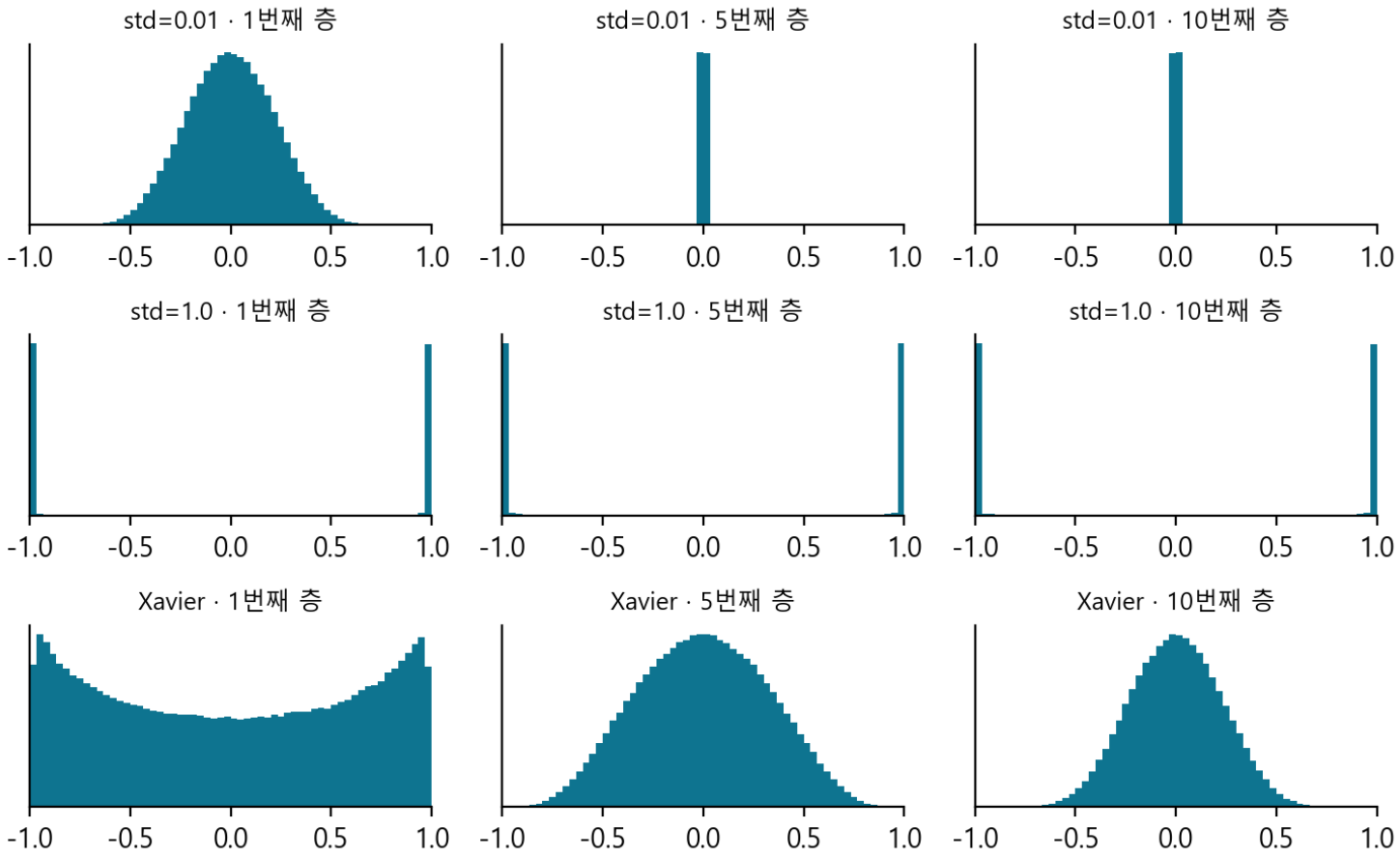
(실제 실행 결과: 씨앗 0, 폭 512, 10층)

설정	10층 후 std
tanh + std 0.01	0.0000003
tanh + std 1.0	0.98 (포화)
tanh + Xavier	0.22
ReLU + std 0.1	79.7 (폭발)
ReLU + Xavier	0.023 (소실)
ReLU + He	0.73 (안정)

읽는 법

로그 눈금이라 세로 한 칸이 10배! 너무 작은 초기값은 몇 층 만에 값이 사라지고, ReLU에는 He가 가장 안정적이다.

실험 결과 ②: tanh 활성값의 분포 (히스토그램)



std = 0.01

층이 깊어질수록 전부 0 한 점으로 모인다 → 소실

std = 1.0

값이 ±1 끝에 달라붙는다(포화) → tanh 기울기 ≈ 0

Xavier

종 모양 분포가 유지된다 → 학습에 좋은 상태

순전파만이 아니라 역전파도: 기울기의 분산

이상적인 초기화는 두 조건을 모두 만족해야 한다:

$$\mathrm{Var}(x^{(l)}) \approx \mathrm{Var}(x^{(l+1)}) \quad \text{(순전파 활성값의 분산이 층을 지나도 유지)}$$

$$\mathrm{Var}(\nabla x^{(l)}) \approx \mathrm{Var}(\nabla x^{(l+1)}) \quad \text{(역전파 기울기의 분산도 층을 지나도 유지)}$$

- $x^{(l+1)}$: (l+1)번째 층의 활성값 $\nabla x^{(l+1)}$: 그 활성값에 대한 손실함수의 기울기
- Var(∇x) : 역전파 중 기울기의 크기가 너무 커지거나 작아지지 않는지 보는 지표
- 분산이 작으면 기울기 소실(앞쪽 층이 학습 안 됨), 크면 기울기 폭발(학습 발산)
- Xavier와 He는 이 두 조건을 만족시키려고 등장했다

실전: 어떤 초기화를 쓸까? (PyTorch)

활성화 함수	권장 초기화	PyTorch 코드
tanh, (sigmoid)	Xavier	nn.init.xavier_normal_(layer.weight)
ReLU, Leaky ReLU	He (Kaiming)	nn.init.kaiming_normal_(w, nonlinearity='relu')
GELU 등	He 계열 또는 기본값	Transformer는 보통 $N(0, 0.02^2)$ 사용

- 편향(bias)은 보통 0으로 초기화한다
- PyTorch의 nn.Linear 기본 초기화는 Kaiming uniform 변형이라, 소규모 실험에서는 별도 설정이 없어도 잘 동작하는 편
- 깊은 네트워크를 직접 만들거나 활성화 함수를 바꿀 때는 초기화도 함께 점검
- ※ sigmoid는 출력 평균이 0.5라서 엄밀히는 '0 대칭'이 아님 → Xavier의 가정에서 조금 벗어남

2교시 정리

- **초기화 목표: 모든 층에서 활성값과 기울기의 분산을 일정하게 유지**
- 분산이 너무 크면 폭발/포화, 너무 작으면 소실 → 깊을수록 효과가 기하급수로 누적
- Var(y) = n_in · Var(w) · Var(x) → Var(w) ≈ 1/n_in 이 기본 원리
- Xavier: 2/(n_in+n_out), tanh 계열 | He: 2/n_in, ReLU 계열
- 실험으로 확인: ReLU + He만 10층 뒤에도 신호가 건강했다

다음 시간 예고

초기화는 '출발'만 안정시킨다. 학습 도중 층 입력 분포가 계속 흔들리는 문제는? → 정규화(Batch Norm, Layer Norm)

3교시

정규화 (Normalization)

내부 공변량 변화 · 미니배치 · Batch Norm · Layer Norm

정규화를 하는 이유

학습 중 활성값은 신경망을 따라 전파된다. 네트워크가 깊을수록 활성값의 분포는 층(layer) 간 또는 배치(batch) 간에 크게 달라질 수 있다.

이 현상 = 내부 공변량 변화

(Internal Covariate Shift)
학습을 불안정하게 만들고 기울기의 크기가 크게 흔들리게 한다.

해결: 정규화 기법

Batch Normalization (배치 정규화)
Layer Normalization (레이어 정규화)
→ 층 입력의 분포를 안정시킨다.

한 줄 요약

정규화 = 각 층으로 들어오는 값의 평균을 0 근처로, 분산을 1 근처로 유지하여 '입력 분포의 변화 폭'을 줄여 주는 것

내부 공변량(Internal Covariate)이란?

공변량(covariate) = 모델의 입력으로 사용되는 변수. 신경망 '안'에서는 이전 층의 출력이 다음 층의 입력이다.



- 데이터 x → 첫 번째 층의 입력
- 첫 번째 층의 출력 h_1 → 두 번째 층의 '내부 공변량'
- 두 번째 층의 출력 h_2 → 세 번째 층의 '내부 공변량'
- 즉, 네트워크 내부에서 각 층이 입력으로 받는 값들을 내부 공변량이라고 부른다

내부 공변량 변화(Internal Covariate Shift)와 문제점

정의

학습이 진행되면서 앞쪽 층의 파라미터(가중치)가 업데이트 되면, 그 결과로 뒤쪽 층이 받는 입력 분포가 계속 변하는 현상.

한 층이 어떤 입력 분포(평균·분산)에 맞춰 학습 중인데 → 이전 층 가중치가 바뀌면서 → 같은 층의 입력 분포가 매 iteration마다 달라진다.

왜 문제인가

· 각 층이 계속 바뀌는 분포에 적응해야 해서 학습이 느려짐
· 활성값이 너무 커지거나 작아져 기울기 폭주/소실 가능
· 학습률을 크게 쓰기 어려워짐
· 초기화에 민감해짐
→ 결과적으로 학습이 불안정해진다

정규화가 하는 일

각 층으로 들어오는 내부 공변량의 평균을 0 근처로, 분산을 1 근처로 유지하도록 만들어 입력 분포의 변화 폭을 줄여준다.
비유: 매번 눈높이가 바뀌는 교실보다, 책상 높이가 일정한 교실에서 공부가 잘 된다.

[최신 자료] 정말로 'Internal Covariate Shift' 때문일까?

- **원논문(Ioffe & Szegedy, 2015)의 설명: BN이 내부 공변량 변화를 줄여서 학습이 빨라진다**
- 후속 연구(Santurkar 외, 2018 'How Does Batch Normalization Help Optimization?')는 BN의 진짜 효과가 ICS 감소보다 손실 지형(loss landscape)을 매끄럽게 만들어 기울기를 예측 가능하게 하는 데 있다고 보고
- 즉, '분포가 안정되어서'보다 '최적화가 쉬운 지형이 되어서'가 더 설득력 있는 설명
- 그래도 실용적 결론은 같다: 정규화를 쓰면 학습이 빠르고 안정적이며 큰 학습률을 쓸 수 있다

강의 방침

교재의 ICS 설명은 직관을 얻는 데 유용하므로 그대로 배우되, 이 개념이 '유일한 정답'은 아니라는 점을 알고 있자.

미니배치(mini-batch)란?

전체 학습 데이터 중에서 한 번의 파라미터 업데이트에 사용되는 작은 데이터 묶음

① 배치 학습 (Batch)

전체 데이터셋을 한 번에 사용

장점: 기울기가 정확함
단점: 메모리 부담 큼, 느림

② 확률적 경사하강법 (SGD)

데이터 1개씩 사용

장점: 빠른 업데이트
단점: 기울기 노이즈 큼, 불안정

③ 미니배치 ★ 가장 많이 사용

일부(예: 32, 64, 128개)만 사용

속도·안정성·메모리의 균형이 좋음

데이터셋 크기: N, 미니배치 크기: m → 한 스텝: $\{(x_1,y_1), \ldots, (x_m,y_m)\}$ 으로 손실 계산 → 기울기 계산 → 가중치 업데이트

Batch Normalization (배치 정규화) ①: 수식

특정 뉴런(또는 채널)의 활성값 미니배치 $\{x_1, x_2, \ldots, x_m\}$ 을 '배치 차원 기준'으로 정규화

$$\mu_B = (1/m) \sum_i x_i$$

(12.1) 미니배치 평균

$$\sigma^2_B = (1/m) \sum_i (x_i - \mu_B)^2$$

(12.2) 미니배치 분산

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$$

(12.3) 정규화: 평균 0, 분산 1

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

(12.4) 배율(γ)·이동(β)으로 복원

- m : 미니배치 안의 샘플 수. μ_B, σ^2_B : 그 미니배치에서 계산한 평균과 분산
- ε(엡실론) : 0으로 나누는 것을 막는 아주 작은 수 (예: 10^{-5})
- γ, β : 학습되는 파라미터 (초기값 γ=1, β=0)

BN 손계산: 값 [2, 4, 6, 8] 한 특징, 배치 크기 4

단계	계산	결과
① 평균	(2+4+6+8)/4	$\mu = 5$
② 분산	$((-3)^2+(-1)^2+1^2+3^2)/4 = 20/4$	$\sigma^2 = 5$
③ 표준편차	$\sqrt{5}$	≈ 2.236
④ 정규화	$(x - 5)/2.236$	−1.342, −0.447, 0.447, 1.342

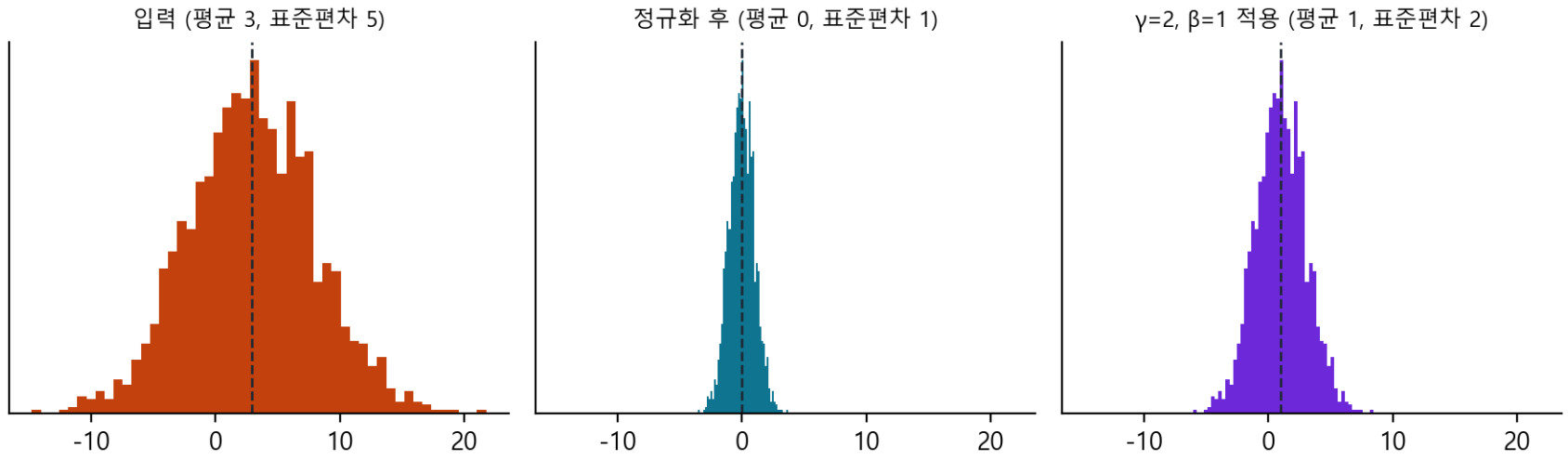
확인

정규화된 값의 평균 = 0
정규화된 값의 분산 = 1
(코드 step3_norm.py 첫 줄 출력과 동일)

γ=2, β=1 을 적용하면

$y = 2 \cdot \hat{x} + 1$
= −1.68, 0.11, 1.89, 3.68
평균 1, 표준편차 2 로 '되돌린' 분포

γ와 β는 왜 필요한가?



- 정규화만 하면 모든 값이 평균 0, 분산 1 이 되어 표현력이 제한된다
- 그래서 모델이 필요하면 다시 스케일을 조절할 수 있도록 γ(배율)와 β(이동)를 '학습 가능한 파라미터'로 둔다
- BN은 학습 안정성을 위해 정규화하고, γ·β는 표현력 회복을 위해 재조정한다 → 이 두 가지가 합쳐져 BatchNorm이 강력해진다

BN을 어디에 넣을까? 그리고 미니배치 크기

일반적으로 선형 변환 이후, 비선형 활성화 이전에 삽입한다:



효과

기울기 흐름을 개선
더 빠른 학습, 더 높은 학습률 사용 가능
(초기화에도 덜 민감 → 실험에서 확인)

BN과 미니배치 크기

BN은 미니배치 내부의 평균·분산을 사용한다.
· 미니배치가 너무 작으면 통계량이 불안정
· 미니배치가 1이면 BN이 의미 없음
→ 작은 배치에서는 LayerNorm, GroupNorm이 선호된다

[보강] 학습할 때와 추론할 때의 BN은 다르다

학습(training) 모드

현재 미니배치의 μ_B, σ^2_B 사용
+ 이동 평균(running mean/var)을 조금씩 갱신
run = 0.9·run + 0.1·(이번 배치 통계)

추론(eval) 모드

배치가 없거나 1개여도 동작해야 한다
→ 학습 중 모아 둔 이동 평균 통계를 고정해서 사용
(PyTorch: model.eval())

- 원본 자료에는 빠져 있지만 실전에서 반드시 알아야 하는 내용
- model.eval()을 잊으면 추론 결과가 배치 구성에 따라 흔들리는 대표적인 버그가 생긴다
- LayerNorm은 샘플 하나 안에서 계산하므로 학습/추론 구분이 없다

직접 구현: BatchNorm (NumPy)

step3_norm.py

```
class BatchNorm:
    def __init__(self, dim, momentum=0.1):
        self.gamma = np.ones(dim)      # 배율 γ (학습됨)
        self.beta  = np.zeros(dim)     # 이동 β (학습됨)
        self.run_mean = np.zeros(dim) # 추론용 이동 평균
        self.run_var  = np.ones(dim)
        self.momentum = momentum
    def __call__(self, x, training=True):
        if training:
            mu  = x.mean(axis=0)       # (12.1) 배치 방향 평균
            var = x.var(axis=0)        # (12.2) 배치 방향 분산
            # 이동 평균 갱신 (추론에서 사용)
            self.run_mean = (1-self.momentum)*self.run_mean + self.momentum*mu
            self.run_var  = (1-self.momentum)*self.run_var  + self.momentum*var
        else:
            mu, var = self.run_mean, self.run_var
        x_hat = (x - mu) / np.sqrt(var + EPS)    # (12.3)
        return self.gamma * x_hat + self.beta    # (12.4)
```

코드 읽는 법

① x 의 모양: (배치, 특징). axis=0 은 '배치 방향' → 특징마다 따로 평균/분산

② γ=1, β=0 으로 시작: 처음에는 순수 정규화

③ training=True : 이번 배치 통계 사용 + 이동 평균 갱신

④ training=False : 저장된 통계 사용 (배치 1개도 OK)

⑤ EPS=1e-5 : 0으로 나누기 방지

Layer Normalization (레이어 정규화)

작은 배치를 쓰거나 시퀀스 모델(NLP, Transformer)에서는 배치 통계가 불안정 → '샘플 하나 안'에서 특징(feature) 차원으로 정규화

$$\mu = (1/d)\,\Sigma_i\, x_i \tag{12.5}$$

$$\sigma^2 = (1/d)\,\Sigma_i\,(x_i - \mu)^2 \tag{12.6}$$

$$\hat{x}_i = (x_i - \mu) \,/\, \sqrt{(\sigma^2 + \varepsilon)} \tag{12.7}$$

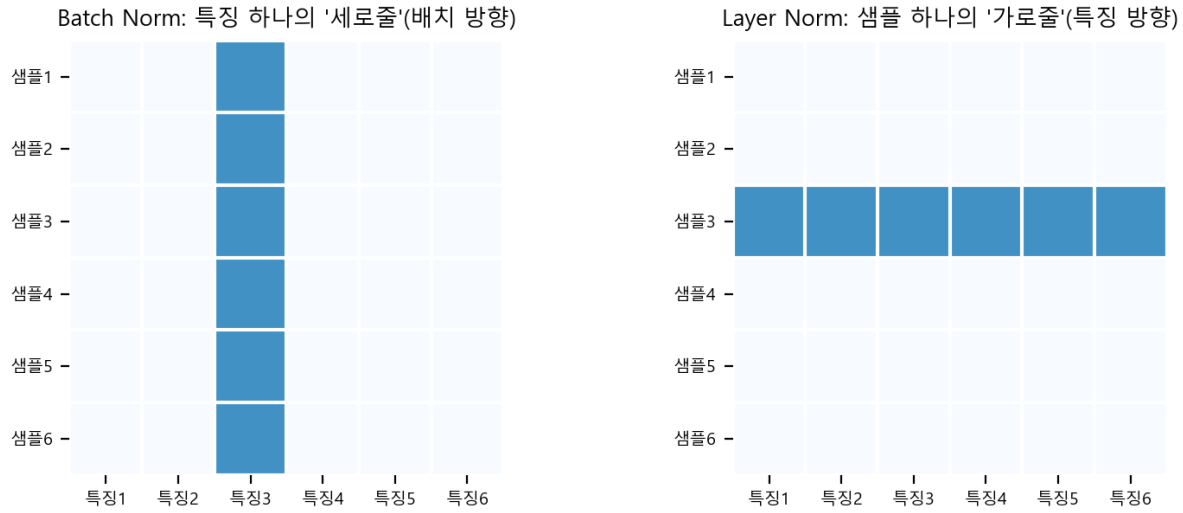
$$y_i = \gamma \cdot \hat{x}_i + \beta \tag{12.8}$$

원본 자료 정정

원본의 식 (12.7)은 μ_B, σ²_B(배치 통계 기호)를 썼다. 레이어 정규화는 샘플 하나의 μ, σ²를 쓰므로 위처럼 바로잡았다.

- d : 샘플 하나의 특징 개수 (벡터 $x \in \mathbb{R}^d$)
- 배치 크기와 무관 → 배치 1개도 OK, 학습/추론이 같다
- RNN, Transformer 같은 순차 모델에 특히 효과적

Batch Norm vs Layer Norm: 어느 방향으로 평균을 낼까?



[그림 12.1 변형] 표 한 장 = 샘플(N) × 특징(C). 파란 부분이 한 번에 평균·분산을 구하는 묶음.

	BatchNorm	LayerNorm
정규화 기준	배치 방향(N)	특징 방향(C,H,W)
통계 계산	특징마다 따로	샘플마다 따로
배치 크기 의존	크다(작으면 불안정)	없다
학습/추론	다름(이동평균)	같음
주 사용처	CNN(이미지)	Transformer, RNN, NLP

한 줄 기억법

BatchNorm = 같은 특징을 '여러 샘플에 걸쳐' 맞춘다 (세로줄)
LayerNorm = 한 샘플을 '자기 특징들끼리' 맞춘다 (가로줄)

직접 구현: LayerNorm + PyTorch로 검증

step3_norm.py

```
class LayerNorm:
    def __init__(self, dim):
        self.gamma = np.ones(dim); self.beta = np.zeros(dim)
    def __call__(self, x):
        mu  = x.mean(axis=-1, keepdims=True)  # (12.5) 샘플별 평균
        var = x.var(axis=-1, keepdims=True)   # (12.6) 샘플별 분산
        x_hat = (x - mu) / np.sqrt(var + EPS) # (12.7)
        return self.gamma * x_hat + self.beta # (12.8)

# --- PyTorch 내장 구현과 비교 ---
ref_b = torch.nn.BatchNorm1d(4, eps=EPS)(t)
ref_l = torch.nn.LayerNorm(4, eps=EPS)(t)
# 출력: BN 일치: True   LN 일치: True
```

코드 읽는 법

① axis=-1 : 마지막 축 = '특징 방향' → 샘플 하나 안에서 계산

② keepdims=True : 모양을 유지해야 (x - mu) 브로드캐스팅이 맞음

③ BatchNorm과 거의 같은 코드, 평균 내는 방향만 다르다!

④ 내장 구현과 결과가 일치 → 구현이 맞다는 증거

출력 확인: BN 출력의 열별 평균 ≈ 0, 표준편차 ≈ 1 / LN 출력의 행별 평균 ≈ 0, 표준편차 ≈ 1

종합 실험: 초기화와 정규화가 학습에 주는 효과

step4_compare.py

```
# 8층 MLP, 나선형 3-클래스 분류, SGD(momentum 0.9)
def build(init, norm, depth=8, width=64):
    for _ in range(depth):
        lin = nn.Linear(d_in, width)
        if init == 'small':  nn.init.normal_(lin.weight, 0, 0.01)
        elif init == 'xavier': nn.init.xavier_normal_(lin.weight)
        else: nn.init.kaiming_normal_(lin.weight, nonlinearity='relu')
        layers.append(lin)              # Linear -> (Norm) -> ReLU
        if norm == 'bn': layers.append(nn.BatchNorm1d(width))
        if norm == 'ln': layers.append(nn.LayerNorm(width))
        layers.append(nn.ReLU())
```

코드 읽는 법

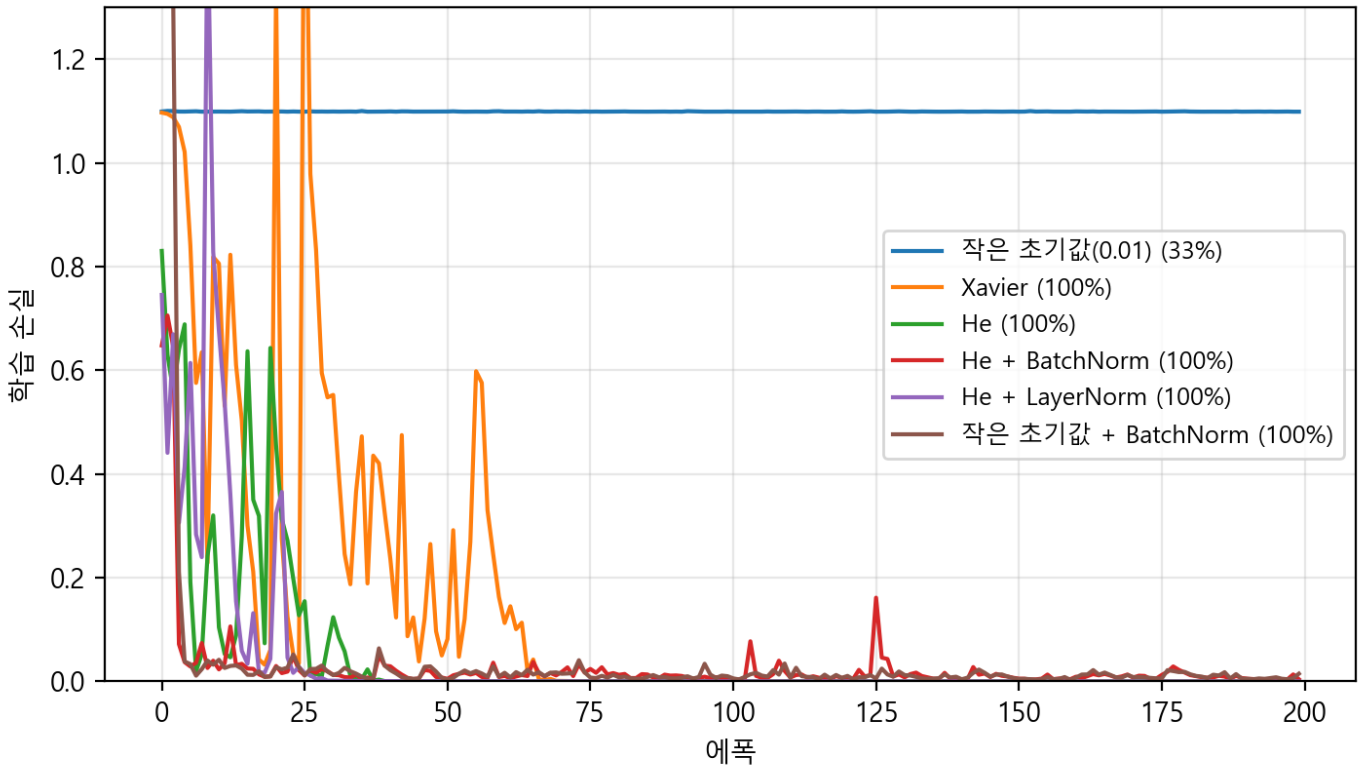
① 같은 모델·같은 데이터에서 초기화(small/Xavier/He)와 정규화(없음/BN/LN)만 바꿔 비교

② 순서: Linear → Norm → ReLU

③ 학습 손실 곡선과 최종 정확도를 비교

예측해 보자: 6가지 설정 중 가장 빨리 배우는 것은? 아예 못 배우는 것은? (다음 장에서 확인)

종합 실험 결과



※ 씨앗 0, 한 번 실행한 결과. 수치는 환경/난수에 따라 달라질 수 있음

설정	10에폭 손실	최종 정확도
작은 초기값(0.01)	1.099	33.3%
Xavier	0.818	100%
He	0.320	100%
He + BatchNorm	0.040	100%
He + LayerNorm	0.828	100%
작은 초기값 + BN	0.032	99.8%

- 작은 초기값만 쓰면 학습이 아예 시작되지 않는다 (손실 1.099 = ln3, 찍기 수준)
- He가 Xavier보다 ReLU에서 빠르다
- BatchNorm을 넣으면 초기화가 나빠도(작은 초기값) 거뜬히 학습 → 초기화 민감도 감소

[최신 자료] Transformer 시대의 정규화

- **Transformer(GPT, BERT, LLaMA 등)는 BatchNorm이 아니라 LayerNorm 계열을 사용: 시퀀스 길이가 달라지고 배치가 작아도 안정적이기 때문**
- RMSNorm : 평균을 빼지 않고 제곱평균제곱근(RMS)으로만 나누는 단순화 버전. LLaMA 등 최신 대형 언어 모델에서 널리 사용 (실습 문제 3)
- Pre-LN : 정규화를 서브층(어텐션/FFN) '앞'에 두는 배치. 깊은 모델이 더 안정적으로 학습됨
- 활성화 함수도 ReLU → GELU / SiLU(Swish) 쪽으로 이동
- 이미지 모델은 여전히 BatchNorm을 많이 사용하고, 작은 배치에서는 GroupNorm을 쓴다

앞서 배운 Transformer 강의의 LayerNorm이 바로 이 수업의 식 (12.5)~(12.8) 입니다!

SUMMARY

Lesson 12 전체 요약

- 심층 신경망은 활성값과 기울기의 스케일·분포에 매우 민감하다
- 정규화(BatchNorm, LayerNorm): 특징 값을 표준화해 불안정을 완화하고 층 사이 정보 흐름을 원활하게 한다 (γ, β로 표현력 회복)
- 적절한 초기화: 학습 초기에 기울기가 너무 크거나 너무 작아지는 것을 방지한다
- Xavier는 대칭 활성화(tanh) 계열, He는 ReLU 계열에 맞춘 초기화
- 정규화와 초기화는 함께 작동하여 현대 딥러닝의 안정적이고 빠르며 견고한 학습의 핵심 기반을 이룬다
- ReLU: 기울기 소실을 줄이고 계산이 빠르지만 Dying ReLU에 주의 → He 초기화·정규화와 함께 쓴다

실습

실습 문제 5개

손계산 · 구현 · 예측 후 실험 (솔루션은 문제 뒤에 이어집니다)

실습 문제 ①~③

문제 1. 손으로 배치 정규화 (γ=1, β=0, ε 무시)

한 특징의 값 x = [1, 3, 5, 7, 9] (배치 크기 5)를 정규화하라. 평균, 분산, 정규화된 5개 값을 구하라.

문제 2. 초기화 표준편차 계산

입력 256개, 출력 128개인 Dense 층에서 Xavier(정규분포형)와 He 초기화의 분산과 표준편차를 각각 구하라.

문제 3. RMSNorm 구현

평균을 빼지 않고 $y = \gamma \cdot x / \sqrt{(\text{mean}(x^2) + \varepsilon)}$ 로 정규화하는 RMSNorm 클래스를 NumPy로 구현하라. x=[[1,2,3,4]]일 때 결과는? LayerNorm과 어떻게 다른가?

실습 문제 ④~⑤

문제 4. 예측 후 실험 (코드: step2_init.py)

ReLU 신경망에서 He 대신 분산 1/n_in (std = √(1/n_in))을 쓰면 10층 뒤 활성값의 표준편차는 He의 결과(≈0.725)와 비교해 어떻게 될까?

먼저 이유와 함께 예측한 뒤, forward_stds로 확인하라.

문제 5. 배치 크기 1의 BatchNorm

step3_norm.py의 BatchNorm(3)에 β=[0.5, 0.5, 0.5]를 설정하고, 샘플 하나 [[10, -3, 7]]을 training=True 로 넣으면 출력은? 왜 그런가? 같은 입력을 LayerNorm(3)에 넣으면 어떻게 다른가?

솔루션

실습 문제 풀이

코드: lesson12/code/practice_solutions.py

솔루션 ①: 손으로 배치 정규화

단계	계산	결과
평균	(1+3+5+7+9)/5	5
분산	$((-4)^2+(-2)^2+0^2+2^2+4^2)/5 = 40/5$	8
표준편차	$\sqrt{8}$	≈ 2.828
정규화 $\hat{x}$	$(x-5)/2.828$	−1.414, −0.707, 0, 0.707, 1.414

검산

새 평균 = (−1.414 −0.707 + 0 + 0.707 + 1.414)/5 = 0, 새 분산 = (2+0.5+0+0.5+2)/5 = 1 ✓

솔루션 ②: 초기화 표준편차

$$\text{Xavier} : \text{Var} = 2/(256+128) = 2/384 \approx 0.00521 \quad \rightarrow \quad \text{std} = \sqrt{0.00521} \approx 0.0722$$

$$\text{He} : \text{Var} = 2/256 = 0.0078125 \quad \rightarrow \quad \text{std} = \sqrt{0.0078125} \approx 0.0884$$

- He의 분산이 더 크다 (ReLU가 절반을 죽이는 것을 보상)
- 코드: np.sqrt(2/(n_in+n_out)), np.sqrt(2/n_in)
- 이 std로 W = rng.normal(0, std, (n_in, n_out)) 를 만들면 초기화 완료

솔루션 ③: RMSNorm

practice_solutions.py

```
class RMSNorm:
    def __init__(self, dim):
        self.gamma = np.ones(dim)
    def __call__(self, x):
        # 제곱의 평균에 루트 = RMS (평균은 빼지 않는다)
        rms = np.sqrt((x ** 2).mean(axis=-1, keepdims=True) + EPS)
        return self.gamma * x / rms

z = np.array([[1., 2, 3, 4]])
# RMSNorm   -> [0.365 0.730 1.095 1.461]
# LayerNorm-> [-1.342 -0.447 0.447 1.342]
```

코드 읽는 법

① RMS = √(제곱의 평균) : 값들의 '평균적인 크기'

② 평균을 빼지 않으므로 부호와 상대 크기가 그대로 보존 (모두 양수)

③ LayerNorm은 평균을 빼서 0 중심으로 옮기지만 RMSNorm은 크기만 맞춘다

④ 계산이 더 간단하고 빨라서 LLaMA 등에서 사용

솔루션 ④: 분산 1/n_in 을 쓰면?

- **예측: ReLU가 출력의 절반을 0으로 만들어 층마다 신호의 분산이 약 1/2(표준편차 약 0.7배)로 줄어든다**
- 결과(10층 뒤 표준편차): He ≈ 0.725 vs 1/n_in ≈ 0.0227 → He가 약 32배 크다
- 층마다 0.7배씩 줄어들며 신호가 점점 소실 → 깊을수록 격차가 커진다
- 이것이 ReLU에 He(2/n_in)가 필요한 이유이며, 그림 '실험 결과 ①'의 ReLU + Xavier 선과 같은 모양이다

```
half = lambda n_in, n_out: np.sqrt(1 / n_in)
forward_stds(relu, half)[-1]   # 0.0227
```

솔루션 ⑤: 배치 크기 1의 BatchNorm

practice_solutions.py

```
bn = BatchNorm(3); bn.beta[:] = [0.5, 0.5, 0.5]
one = np.array([[10., -3, 7]])
bn(one, training=True)    # [[0.5 0.5 0.5]]  <- β만 남는다!
LayerNorm(3)(one)         # [[ 0.96 -1.379  0.42 ]]
```

- 배치에 샘플이 1개면 '배치 평균 = 자기 자신' → x − μ = 0 → 정규화 값이 0 → 출력은 β만 남는다 (입력 정보가 전부 사라짐)
- 그래서 BatchNorm은 충분히 큰 배치가 필요하다
- LayerNorm은 샘플 안의 특징끼리 계산하므로 샘플 1개여도 정상 동작한다
- 추론 때(training=False)는 저장된 이동 평균 통계를 쓰므로 배치 1개여도 괜찮다

APPENDIX

원본 자료 검토 결과 (강사용): 바로잡은 내용

원본 위치	문제	이 강의에서의 처리
BN 식 설명 (12.3)	'평균을 빼는 것이 분산을 줄인다'고 설명 — 평균을 빼도 분산은 변하지 않음 (분산을 줄이는 것은 표준편차로 나누는 단계)	평균 빼기=0 중심 이동, 표준편차로 나누기=스케일 1로 구분해 설명
LN 식 (12.7)	레이어 정규화 식에 배치 통계 기호 μ_B, σ^2_B 사용	μ, σ^2로 정정
Xavier 설명	1/n_in 과 2/(n_in+n_out) 를 같은 방법처럼 병기 / sigmoid를 '대칭'으로 설명	LeCun·Glorot 구분, sigmoid 주의 추가
'분산이 크면 폭발'	tanh에서는 폭발이 아니라 포화(±1)로 나타남	실험으로 포화·폭발 구분(ReLU)
BN 설명 전반	추론 시 이동 평균 사용 설명 누락, ICS 설명이 최신 연구와 다름	[보강]·[최신 자료] 슬라이드 추가
표기·중복	Internal Covariants, Intialization, sparity, Parameteric 오타. 개요와 '정규화를 하는 이유' 문단 중복	바로잡음 / 통합

※ Lesson 13(데이터 증강)은 이 강의 범위가 아니라 제외했다.

수고하셨습니다!

다음 시간: Lesson 13. Data Augmentation (데이터 증강)

튜링랩 프로그래밍학원 | TuringLab. Programming Academy